MANUAL DE USUARIO

Proyecto 1: Contacto 3xtrat3rr3str3D

Fase 2

Enmer Oswaldo Sandoval Mazariegos
3301795511201

¿QUÉ HACE EL SISTEMA?

El sistema es un compilador para tres lenguajes: Y?, Zetariano y Pig Latin. Revisa el código de forma léxica, sintáctica y semántica, y si no hay errores genera las cuartetas, el código de tres direcciones en C (programa.c) y el código RISC-V (programa.s).

El código RISC-V se puede ejecutar dentro del mismo IDE con un simulador propio, sin instalar programas externos.

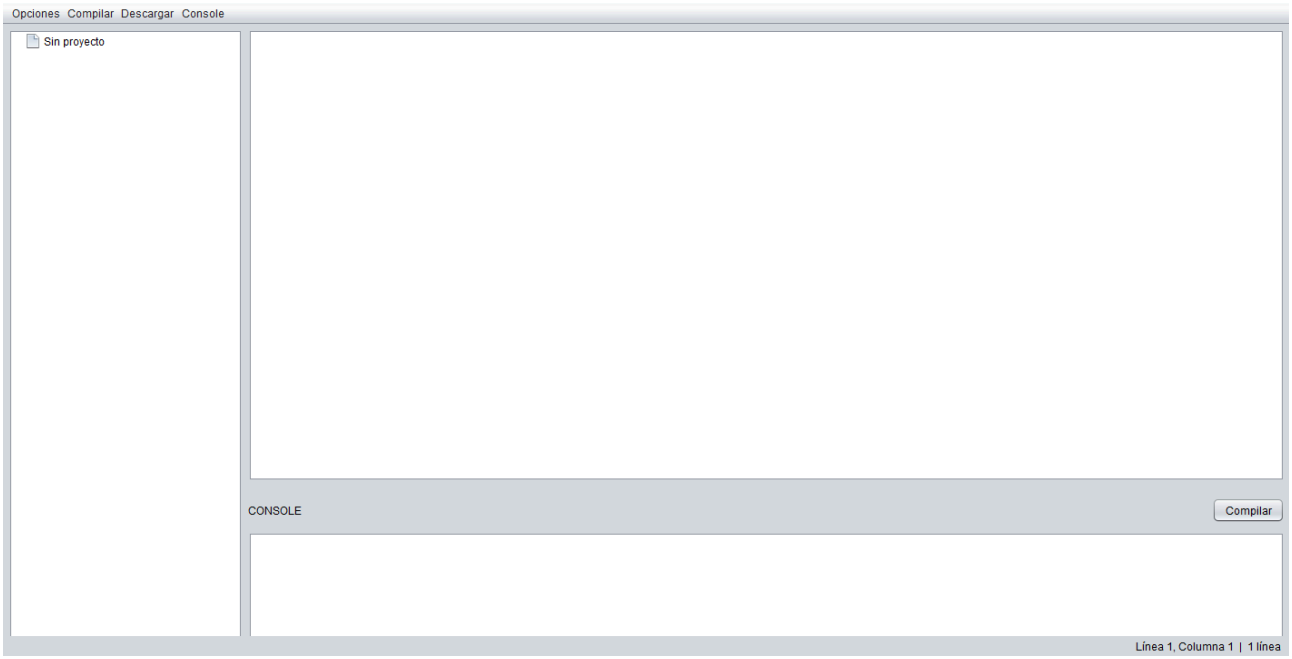
Un proyecto tiene un archivo .pig principal (Pig Latin) que importa clases de Zetariano (.z) y funciones de Y? (.y).

REQUISITOS Y CÓMO INICIAR EL PROGRAMA

- Java 17 o superior.
- Opcional: gcc, si se quiere compilar el programa.c generado. Si no está instalado, el IDE lo indica en la consola y el resto funciona igual.

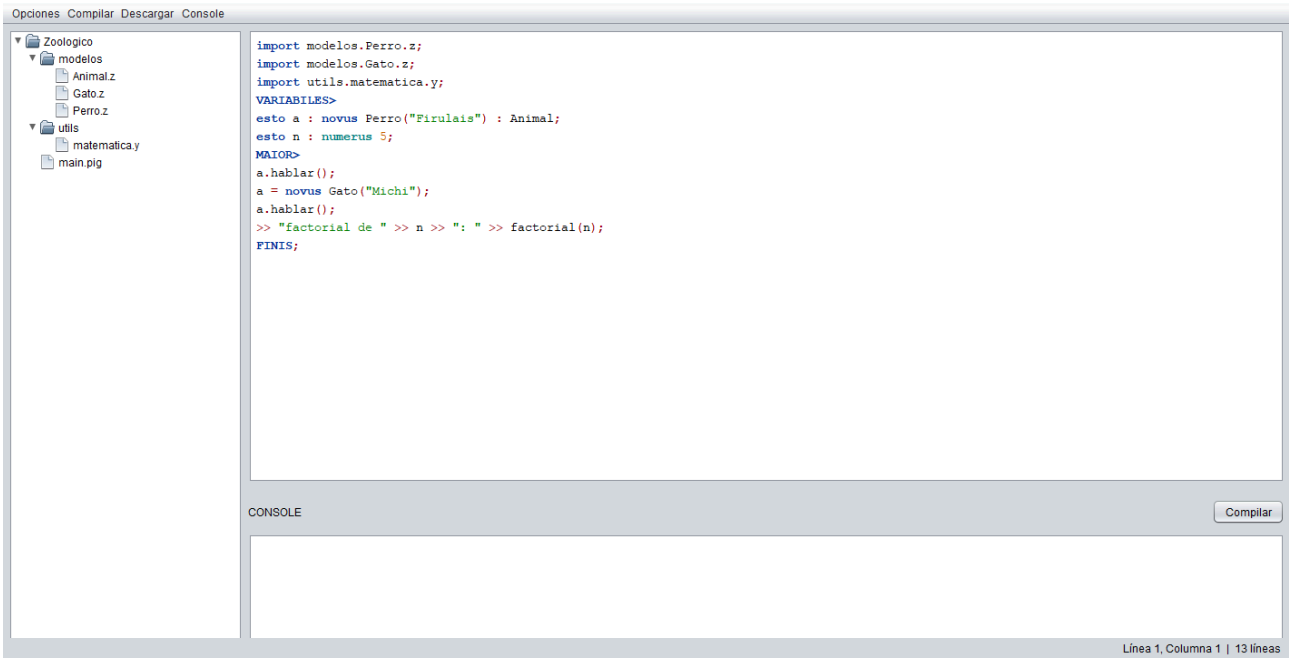
Para abrir el IDE se ejecuta el archivo JAR:

```
java -jar Proyecto_Fase1.jar
```



IDE al iniciar

LA INTERFAZ

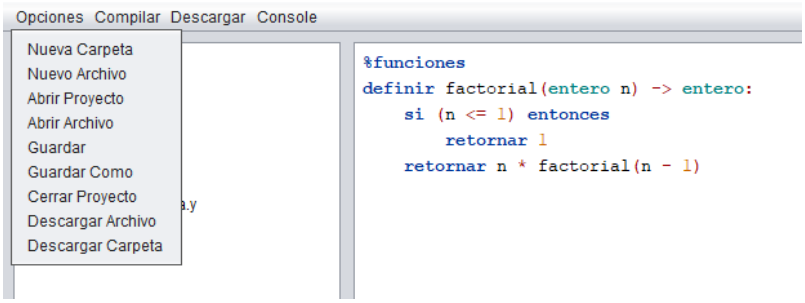


IDE con un proyecto abierto

- Árbol de archivos: muestra las carpetas y archivos del proyecto. Con un clic sobre un archivo se abre en el editor.
- Editor: donde se escribe el código. Se colorea según el lenguaje del archivo abierto.
- Consola: muestra los mensajes del IDE, los errores encontrados y la salida del programa cuando se ejecuta en el simulador.
- Botón Compilar: compila el proyecto abierto.
- Barra inferior: muestra la línea y la columna donde está el cursor y el total de líneas del archivo.
- Título de la ventana: muestra el nombre del archivo abierto. Si tiene cambios sin guardar aparece un *.

MENÚS

Opciones

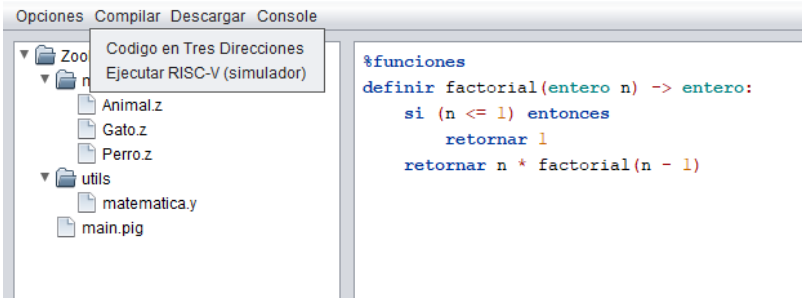


Menú Opciones

- Nueva Carpeta: si no hay proyecto abierto, crea un proyecto nuevo. Si ya hay uno, crea una subcarpeta dentro de la carpeta seleccionada en el árbol.
- Nuevo Archivo: crea un archivo .pig, .y o .z dentro de la carpeta seleccionada.
- Abrir Proyecto: abre una carpeta como proyecto y la carga en el árbol.

- Abrir Archivo: abre un archivo suelto en el editor.
- Guardar y Guardar Como: guardan el archivo abierto, o lo guardan con otro nombre o en otro lugar.
- Cerrar Proyecto: cierra el proyecto. Si hay cambios sin guardar, pregunta antes.
- Descargar Archivo y Descargar Carpeta: copian el archivo o la carpeta seleccionada al lugar que usted elija.

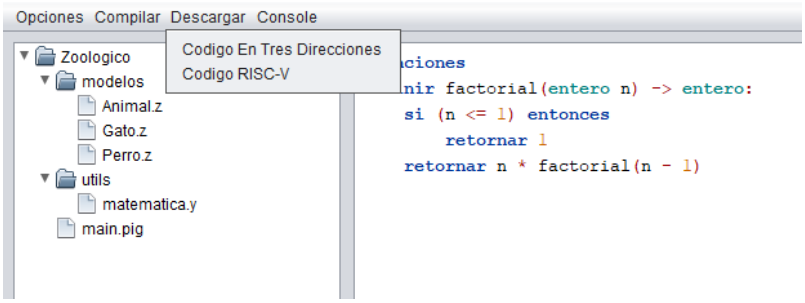
Compilar



Menú Compilar

- Codigo en Tres Direcciones: compila el proyecto. Hace lo mismo que el botón Compilar.
- Ejecutar RISC-V (simulador): ejecuta en el simulador el último programa compilado sin errores.

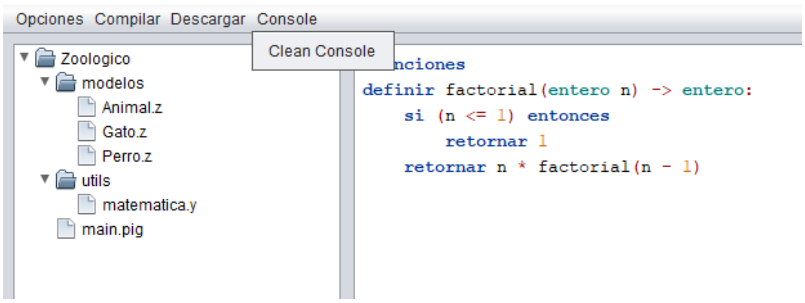
Descargar



Menú Descargar

- Codigo En Tres Direcciones: guarda el programa.c del último programa compilado donde usted elija.
- Codigo RISC-V: guarda el programa.s del último programa compilado donde usted elija.

Console



Menú Console

- Clean Console: borra todo el texto de la consola.

PROYECTOS Y ARCHIVOS

Crear un proyecto

1. Sin ningún proyecto abierto, elija Opciones, Nueva Carpeta.
2. Seleccione la carpeta donde se va a crear el proyecto y escriba su nombre.
3. El IDE crea la carpeta con un main.pig de ejemplo y lo abre en el editor.

Crear archivos

Con Opciones, Nuevo Archivo, se escribe el nombre con su extensión. El archivo se crea en la carpeta seleccionada en el árbol con una plantilla inicial:

- .pig: secciones VARIABILES>, MAIOR> y FINIS; con un "Hola mundo".
- .y: sección %funciones con una función de ejemplo.
- .z: una clase pública con el mismo nombre del archivo y su constructor vacío.

Si la extensión no es .pig, .y o .z, o ya existe un archivo con ese nombre, el IDE muestra un aviso y no crea nada.

Estructura de un proyecto

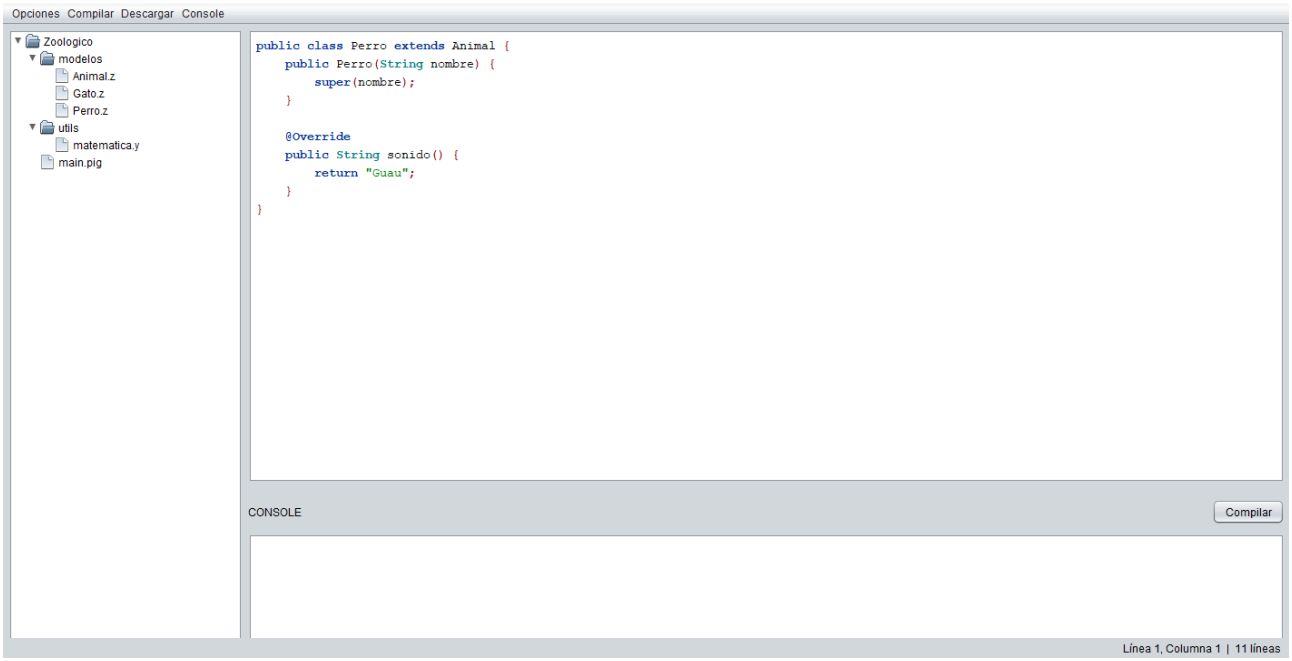
El archivo .pig importa los demás archivos con rutas relativas a la carpeta del proyecto:

```
import modelos.Perro.z;
import modelos.Gato.z;
import utils.matematica.y;
```

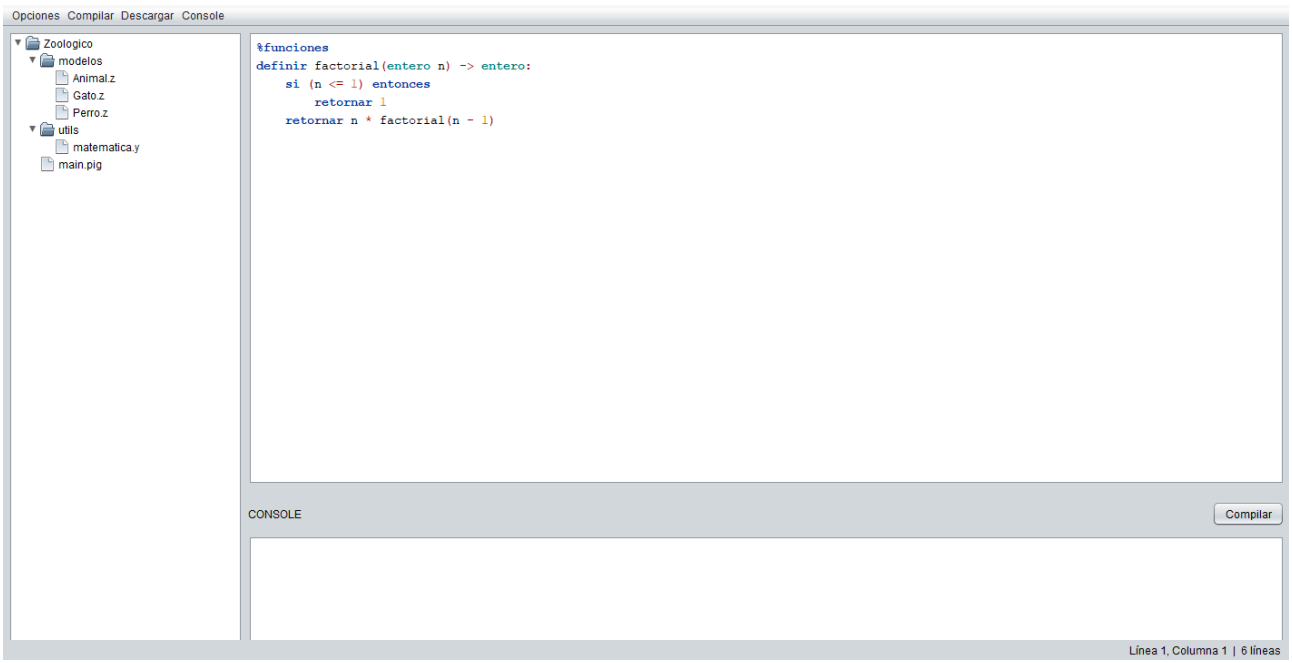
Si una clase hereda de otra, no hace falta importar la clase padre: el compilador la busca primero en la misma carpeta y luego en todo el proyecto.

EDITOR CON COLORES

El código se colorea mientras se escribe. Cada lenguaje tiene sus propias palabras reservadas y colores.



Clase de Zetariano



Funciones de Y?

COMPILAR

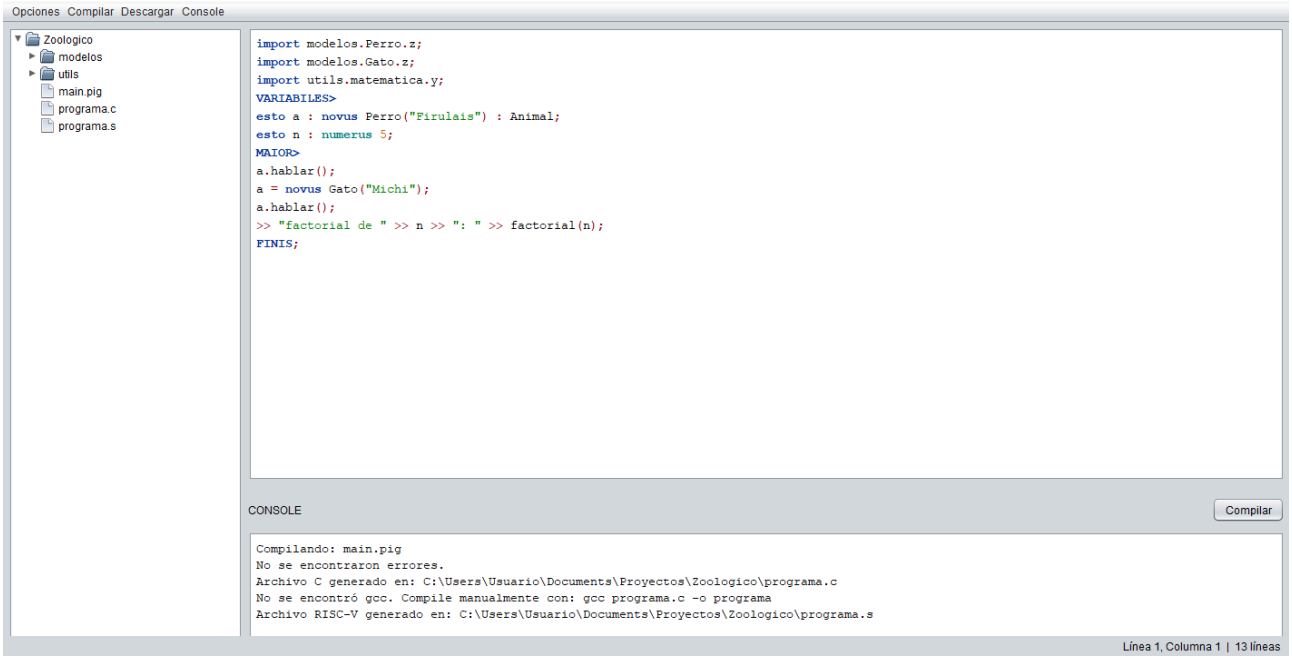
Se compila con el botón Compilar o con Compilar, Codigo en Tres Direcciones. Antes de compilar, el IDE guarda el archivo abierto.

El IDE compila el .pig que esté abierto en el editor. Si el archivo abierto no es .pig, busca el main.pig del proyecto o el primer .pig de la carpeta.

Si no hay errores:

- Se guardan programa.c y programa.s en la carpeta del proyecto y aparecen en el árbol.
- Si gcc está instalado, el IDE también compila programa.c y genera el ejecutable programa.

- Se abre la ventana de Reportes de compilación.



Consola después de compilar

REPORTES DE COMPILACIÓN

La ventana de reportes tiene una pestaña por cada reporte:

- Tabla de símbolos: cada variable, parámetro, atributo, método, constructor, función y clase del programa, con su tipo, categoría, ámbito, posición en memoria y archivo donde se declaró.
- Tabla de tipos: los tipos que usa el programa, su categoría, cuántas celdas ocupa, cómo se guarda en memoria y, en las clases, la posición de cada atributo.
- Compatibilidad: el resultado de cada operación entre dos tipos. La columna Lenguaje dice "todos" cuando la regla es igual en los tres lenguajes, o el nombre del lenguaje cuando solo aplica en uno. En el operador =, el tipo izquierdo es la variable y el derecho es el valor que se le asigna.
- Cuartetas: el código intermedio con operador, argumento 1, argumento 2 y resultado.
- Código de tres direcciones: el código C que se guarda como programa.c.
- RISC-V: el código ensamblador que se guarda como programa.s.

Tabla de símbolos | Tabla de tipos | Compatibilidad | Cuartetas | Código de tres direcciones | RISC-V

Nombre	Tipo	Categoría	Ámbito	Posición	Archivo
Perro	Perro	clase	global	-	modelos/Perro.z
Animal	Animal	clase	global	-	modelos/Animal.z
Gato	Gato	clase	global	-	modelos/Gato.z
factorial(entero)	entero	función	global	-	utils/matematica.y
nombre	cadena	atributo	Animal	1	modelos/Animal.z
patas	entero	atributo	Animal	2	modelos/Animal.z
Animal(cadena)	vacio	constructor	Animal	-	modelos/Animal.z
sonido()	cadena	método	Animal	-	modelos/Animal.z
hablar()	vacio	método	Animal	-	modelos/Animal.z
Perro(cadena)	vacio	constructor	Perro	-	modelos/Perro.z
sonido()	cadena	método	Perro	-	modelos/Perro.z
Gato(cadena)	vacio	constructor	Gato	-	modelos/Gato.z
sonido()	cadena	método	Gato	-	modelos/Gato.z
n	entero	parámetro	factorial	1	utils/matematica.y
nombre	cadena	parámetro	Animal.Animal	2	modelos/Animal.z
nombre	cadena	parámetro	Perro.Perro	2	modelos/Perro.z
nombre	cadena	parámetro	Gato.Gato	2	modelos/Gato.z
a	Animal	variable	main	0	main.pig
n	entero	variable	main	1	main.pig

Tabla de símbolos

Tabla de símbolos | Tabla de tipos | Compatibilidad | Cuartetas | Código de tres direcciones | RISC-V

Tipo	Categoría	Tamaño	Cómo se guarda	Miembros (posición)
entero	primitivo	1	valor directo en la celda	
decimal	primitivo	1	valor directo en la celda	
caracter	primitivo	1	valor directo en la celda	
booleano	primitivo	1	valor directo en la celda	
cadena	primitivo	1	puntero a los caracteres en el heap	
Perro	clase, hereda de Animal	3	puntero a un bloque de 3 celdas en el heap (celda 0: id de la clase)	nombre(1), patas(2)
Animal	clase	3	puntero a un bloque de 3 celdas en el heap (celda 0: id de la clase)	nombre(1), patas(2)
Gato	clase, hereda de Animal	3	puntero a un bloque de 3 celdas en el heap (celda 0: id de la clase)	nombre(1), patas(2)

Tabla de tipos

Tabla de símbolos | Tabla de tipos | Compatibilidad | Cuartetas | Código de tres direcciones | RISC-V

Operador	Tipo izquierdo	Tipo derecho	Resultado	Lenguaje
+	entero	entero	entero	todos
+	entero	decimal	decimal	todos
+	entero	caracter	entero	todos
+	entero	cadena	cadena	todos
+	decimal	entero	decimal	todos
+	decimal	decimal	decimal	todos
+	decimal	caracter	decimal	todos
+	decimal	cadena	cadena	todos
+	caracter	entero	entero	todos
+	caracter	decimal	decimal	todos
+	caracter	caracter	cadena	Y?, Pig Latin
+	caracter	caracter	entero	Zetariano
+	caracter	cadena	cadena	todos
+	booleano	cadena	cadena	todos
+	cadena	entero	cadena	todos
+	cadena	decimal	cadena	todos
+	cadena	caracter	cadena	todos
+	cadena	booleano	cadena	todos
+	cadena	cadena	cadena	todos
- * /	entero	entero	entero	todos
- * /	entero	decimal	decimal	todos
- * /	entero	caracter	entero	todos
- * /	decimal	entero	decimal	todos
- * /	decimal	decimal	decimal	todos
- * /	decimal	caracter	decimal	todos
- * /	caracter	entero	entero	todos
- * /	caracter	decimal	decimal	todos
- * /	caracter	caracter	entero	todos
%	entero	entero	entero	Zetariano

Compatibilidad

Tabla de símbolos | Tabla de tipos | Compatibilidad | Cuartetas | Código de tres direcciones | RISC-V

#	Operador	Argumento 1	Argumento 2	Resultado
0	inicio	nativeImprimirCadena		
1	+	P	1	t0
2	=	stack[(int) t0]		t1
3	etiqueta			et1
4	=	heap[(int) t1]		t2
5	if ==	t2	0	et2
6	print	t2	%c	
7	+	t1	1	t1
8	goto			et1
9	etiqueta			et2
10	etiqueta			et0
11	-	P	1	t3
12	=	stack[(int) t3]		t4
13	if ==	t4	65	et65
14	if ==	t4	94	et94
15	if ==	t4	95	et95
16	fin	nativeImprimirCadena		
17	inicio	nativeConcatenar		
18	+	P	1	t5
19	=	stack[(int) t5]		t6
20	+	P	2	t7
21	=	stack[(int) t7]		t8
22	=	H		t9
23	etiqueta			et4
24	=	heap[(int) t6]		t10
25	if ==	t10	0	et5
26	=	t10		heap[(int) H]
27	+	H	1	H
28	if <	H	1000000	et6

Cuartetas

Tabla de símbolos | Tabla de tipos | Compatibilidad | Cuartetas | Código de tres direcciones | RISC-V

```
#include <stdio.h>

double stack[100000];
double heap[1000000];
int P = 0;
int H = 1;

double t0, t1, t2, t3, t4, t5, t6, t7, t8, t9;
double t10, t11, t12, t13, t14, t15, t16, t17, t18, t19;
double t20, t21, t22, t23, t24, t25, t26, t27, t28, t29;
double t30, t31, t32, t33, t34, t35, t36, t37, t38, t39;
double t40, t41, t42, t43, t44, t45, t46, t47, t48, t49;
double t50, t51, t52, t53, t54, t55, t56, t57, t58, t59;
double t60, t61, t62, t63, t64, t65, t66, t67, t68, t69;
double t70, t71, t72, t73, t74, t75, t76, t77, t78, t79;
double t80, t81, t82, t83, t84, t85, t86, t87, t88, t89;
double t90, t91, t92, t93, t94, t95, t96, t97, t98, t99;
double t100, t101, t102, t103, t104, t105, t106, t107, t108, t109;
double t110, t111, t112, t113, t114, t115, t116, t117, t118, t119;
double t120, t121, t122, t123, t124, t125, t126, t127, t128, t129;
double t130, t131, t132, t133, t134, t135, t136, t137, t138, t139;
double t140, t141, t142, t143, t144, t145, t146, t147, t148, t149;
double t150, t151, t152, t153, t154, t155, t156, t157, t158, t159;
double t160, t161, t162, t163, t164, t165, t166, t167, t168, t169;
double t170, t171, t172, t173, t174, t175, t176, t177, t178, t179;
double t180, t181, t182, t183, t184, t185, t186, t187, t188, t189;
double t190, t191, t192, t193, t194, t195, t196, t197, t198, t199;
```

Código de tres direcciones

Tabla de símbolos | Tabla de tipos | Compatibilidad | Cuartetas | Código de tres direcciones | RISC-V

```
.data
stack: .space 800000
heap: .space 8000000
temporales: .space 2336
str0: .string "Error: heap agotado\n"
str1: .string "\nError en tiempo de ejecución: se usó un objeto nulo en la línea 8, columna 3\n"
str2: .string "\nError en tiempo de ejecución: se usó un objeto nulo en la línea 10, columna 3\n"
str3: .string "\n"
.align 3
dbl0: .double 0.5
dbl1: .double 0.1

.text
.globl main
main:
    la s0, stack
    la s1, heap
    la s2, temporales
    fcvt.d.l fs0, zero
    li t0, 1
    fcvt.d.l fs1, t0
    fmv.d ft0, fs1
    fsd ft0, 1656(s2)
    li t1, 70
    fcvt.d.l ft0, t1
    fmv.d ft3, fs1
    fcvt.l.d t0, ft3, rtz
```

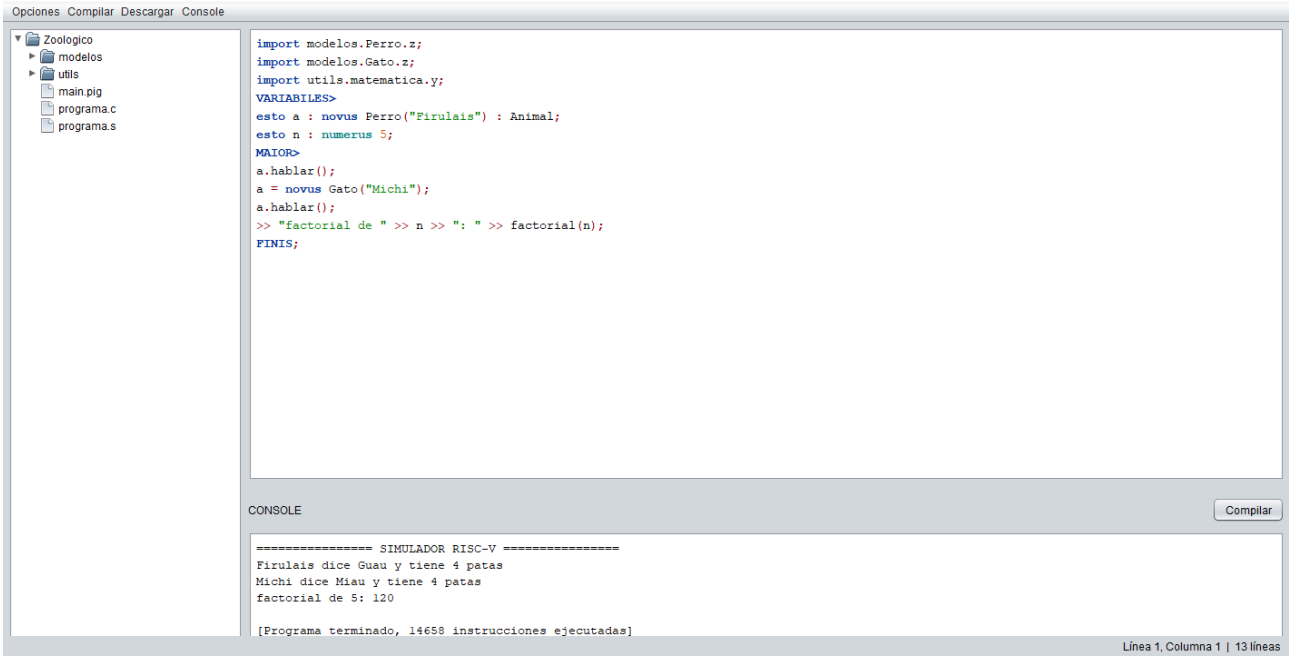
RISC-V

EJECUTAR EN EL SIMULADOR RISC-V

Después de compilar sin errores, elija Compilar, Ejecutar RISC-V (simulador). El programa se ejecuta en el simulador del IDE y su salida aparece en la consola. Al terminar se indica cuántas instrucciones se ejecutaron.

- Si el programa lee datos, el IDE abre una ventana llamada Lectura para escribir la entrada.
- Si ocurre un error durante la ejecución, se muestra en la consola y el programa termina. Por ejemplo: "Error: heap agotado" o "Error en tiempo de ejecución: se usó un objeto nulo en la línea 8, columna 3".

- Mientras un programa se está ejecutando no se puede iniciar otro.



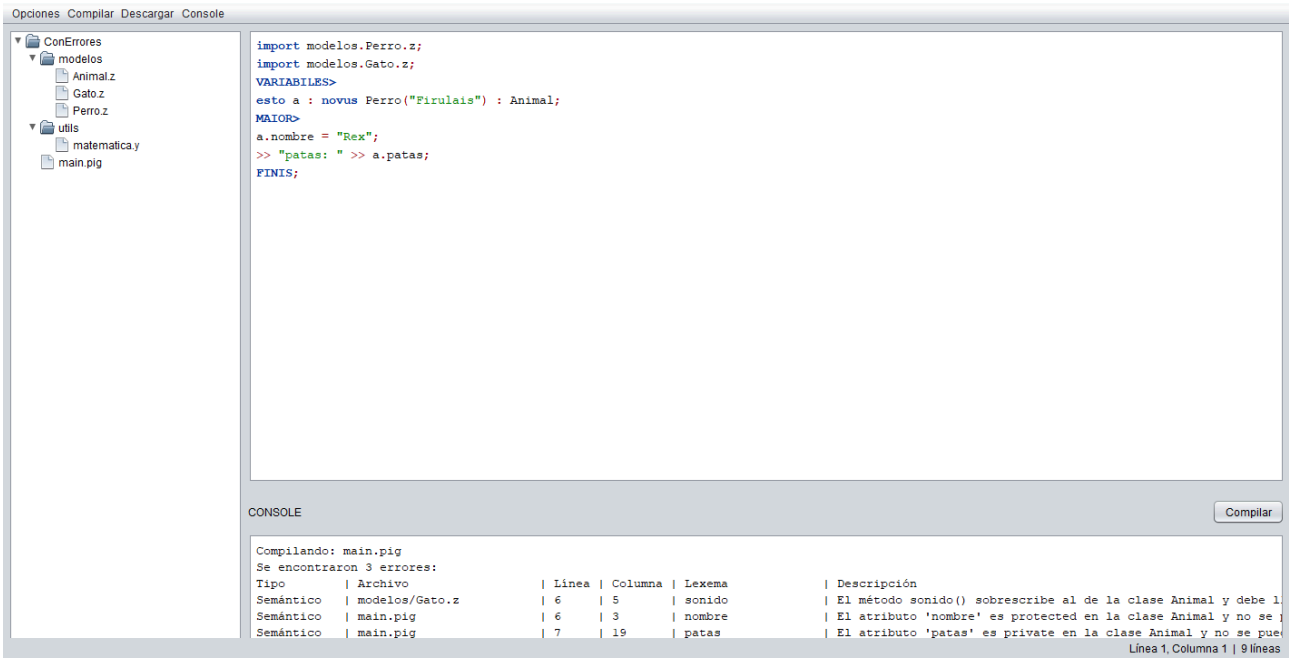
Salida en el simulador

ERRORES

Si el código tiene errores, no se genera código. La consola muestra la lista de errores y se abre la ventana Reporte de errores con el tipo de error (léxico, sintáctico o semántico), el archivo, la línea, la columna, el lexema y la descripción.

Tipo	Archivo	Línea	Columna	Lexema	Descripción
Semántico	modelos/Gato.z	6	5	sonido	El método sonido() sobrescribe al de la clase Animal y debe llevar la anotación @Override
Semántico	main.pig	6	3	nombre	El atributo 'nombre' es protected en la clase Animal y no se puede usar desde aquí
Semántico	main.pig	7	19	patas	El atributo 'patas' es private en la clase Animal y no se puede usar desde aquí

Reporte de errores



Errores en la consola

DESCARGAS

- Descargar, Codigo En Tres Direcciones: el programa.c del último programa compilado.
- Descargar, Codigo RISC-V: el programa.s del último programa compilado.
- Opciones, Descargar Archivo: el archivo seleccionado en el árbol, o el que está abierto en el editor.
- Opciones, Descargar Carpeta: la carpeta seleccionada en el árbol con todo su contenido.

En todos los casos se abre una ventana para elegir dónde guardar. Las descargas de código solo funcionan después de compilar un programa sin errores.

REGLAS DE CLASES EN ZETARIANO

- Herencia: solo herencia simple. Una clase hereda de una sola clase con extends.
- @Override: es obligatorio al sobrescribir un método del padre, y es error ponerlo en un método que no sobrescribe nada. El método sobrescrito puede retornar el mismo tipo o una subclase, y no puede ser menos visible que el del padre.
- super: super(...) llama al constructor del padre y debe ser la primera instrucción del constructor. super.metodo() llama a la versión del padre.
- Constructor del padre: si el constructor del hijo no llama a super, se llama solo al constructor vacío del padre. Si el padre no tiene constructor vacío, es obligatorio llamar a super(...).
- public: se puede usar desde cualquier lugar.
- protected: solo dentro de la clase y de sus clases hijas.
- private y sin modificador: solo dentro de la propia clase. Las clases hijas tampoco lo ven.
- Polimorfismo: con esto a : novus Perro("Firulais") : Animal; la llamada a.hablar() ejecuta el método de Perro si lo sobrescribe.

USO DESDE LA CONSOLA

El JAR también se puede usar sin abrir el IDE, pasándole el archivo .pig que se quiere compilar:

```
java -jar Proyecto_Fase1.jar main.pig [opciones]
```

- --ast: muestra el árbol de sintaxis.
- --tipos: muestra la tabla de tipos y la tabla de compatibilidad.
- --cuartetas: muestra las cuartetas.
- --c3d: muestra el código de tres direcciones en C.
- --riscv: muestra el código RISC-V.
- --ejecutar: ejecuta el programa en el simulador RISC-V.

Siempre se muestran la tabla de símbolos y los errores. Si no hay errores, programa.c y programa.s se guardan en la carpeta desde donde se ejecutó el comando.

Ejemplo:

```
java -jar Proyecto_Fase1.jar main.pig --cuartetas --ejecutar
```